

PRD Addendum — Modisoft Demand & Sales Forecast v1.3

Owner: Ebuka (AI PM) **Developer:** Nishant (Lead Dev — Demand Forecasting) **Date:** February 27, 2026 **Status:** Frontend prototype complete — Backend implementation required

Purpose of This Document

This addendum is a **continuation of the original PRD** (Sections 1–16, ending at v1.2 in the Change Log). It documents **all feature changes, additions, and removals** implemented in the frontend prototype since v1.2, along with the reasoning behind each decision, the technical requirements for backend implementation, and estimated complexity for Nishant to plan sprints.

How to read this: *Each feature below includes:*

- 1. What changed** — *The frontend behavior that now exists*
- 2. Why** — *The UX / business reasoning*
- 3. What Nishant needs to build** — *APIs, data models, ML/AI requirements*
- 4. Estimated complexity** — *T-shirt size (S / M / L / XL)*

Table of Contents

1. REMOVED: Lock Plan / Execution Board
2. REMOVED: Scenario Inputs & Multiplier Engine
3. ADDED: Inline Action Center (Suggested Actions)
4. ADDED: Action Engine (actionEngine.ts)
5. ADDED: Action Next-Step Modal (Task Delegation)
6. ADDED: Quick Order Modal (Grocery/Retail)
7. ADDED: Substitute Modal (Grocery/Retail)
8. ADDED: New Item Simulator
9. ADDED: Fuel Forecasting Module (Convenience)

10. EXISTING (Updated): Categories at Risk Widget
 11. New API Contracts
 12. New Data Models / Tables
 13. AI / ML Requirements
 14. Third-Party Services & Tools
 15. Updated Acceptance Criteria
 16. Migration Notes
 17. Change Log Entry
-

1. REMOVED: Lock Plan / Execution Board

What was removed

The entire `ExecutionBoard.tsx` component and its associated types (`ApprovedPlan` , `PacingDay` , `WeeklyReviewSummary` , `PlanStatus` , `ScenarioInputs`) have been **deleted** from the codebase.

This was a multi-step enterprise workflow:

- **Step 1:** User views forecast → adjusts scenario sliders
- **Step 2:** User "locks" the plan → system creates an `ApprovedPlan`
- **Step 3:** Pacing tracker shows daily progress against the locked plan
- **Step 4:** Weekly review summarizes how reality compared to the plan

Why it was removed

- **Too complex for our SMB persona.** Our primary user is a 50–65 year old store owner who wants "what's coming and what to do today." They are not project managers who will lock plans, track pacing metrics, or run weekly reviews.
- **Too many clicks before value.** The old flow required 4 steps (Forecast → Approve → Pace → Review) before any action was taken. The new flow is: see action → do action.
- **Redundancy.** The Action Center (see Section 3) now covers the "what to do" use case in a more direct, low-friction way.

What Nishant needs to do

- **Nothing.** No backend was built for this feature. The types and component only existed in the frontend prototype. Nishant should **not** build any plan-locking, pacing, or weekly-review APIs.
- If any database migration scripts reference `approved_plans` or `pacing_days` tables, they should be removed.

Estimated complexity: N/A (deletion only)

2. REMOVED: Scenario Inputs & Multiplier Engine

What was removed

- The `ScenarioInputs` interface (staffingMultiplier, promoAggressiveness, inventoryBuffer, pricingStrategy)
- The `calculateScenarioMultiplier()` function in `actionEngine.ts`
- The scenario multiplier was previously applied to action generation, inflating/deflating recommendations based on how aggressively the user configured sliders

Why it was removed

- **Users don't think in multipliers.** Asking an SMB owner to set "promoAggressiveness: 1.3" or "inventoryBuffer: 1.5" is confusing. These are analyst-level controls.
- **Replaced by direct action.** Instead of tweaking abstract sliders, users now see concrete actions ("Run promo for Heat Wave" or "Reduce hours on Thursday") and either do them or skip them.
- **Scenario editing (promo/price per item) still exists** in the Top Items Table (Section 6.4 of the original PRD). That inline editing is the right level of simplicity.

What Nishant needs to do

- **Nothing.** Do not build a scenario multiplier API. The existing `POST /api/scenario { sku_id, promo_flag, price_change }` from the original PRD (Section 7.5) is sufficient for item-level what-if analysis.

Estimated complexity: N/A (deletion only)

3. ADDED: Inline Action Center (Suggested Actions)

What it is

A card embedded **directly in the main dashboard** (not behind a button or drawer) titled **"Suggested Actions Based on Forecast"**. It sits below the charts and above the business-type widgets.

UI behavior:

- Shows a badge with the count of pending actions
- Displays total potential dollar value across all open actions (e.g., "\$4,200 potential")
- Filter tabs: All | Open | Ignored | Done
- Each action card shows:
 - Priority badge (High / Medium / Low) with color coding
 - Action type icon (labor, promo, pricing, event, fuel)
 - Title and description
 - Expected value label (e.g., "+\$1,200 revenue" or "Save 4 labor hours")
 - Due date
 - "Take Action" button → opens the Action Next-Step Modal (Section 5)
 - "Skip" button → marks as ignored
- Actions that are "done" show a green checkmark with the completed date

Why it was built this way

- **Zero-click visibility.** The owner sees what needs attention the moment they open the dashboard. No need to click a button or open a drawer.
- **Replaces both the old "Lock Plan" button AND the old "Action Center" drawer.** We had a brief period where both existed, creating redundancy. Now there is exactly one place to see and act on recommendations.
- **Aligns with PRD Section 6.3 principle:** "Actionable widgets by business type."

What Nishant needs to build

API: **GET /api/actions**

```
GET /api/actions?store_id={store_id}&business_type={type}&date={YYYY-MM-DD}
```

```
Response: {  
  actions: ActionItem[],  
  stats: {  
    openCount: number,  
    acceptedCount: number,  
    doneCount: number,  
    ignoredCount: number,  
    totalExpectedValue: number,  
    completedValue: number  
  }  
}
```

API: **PATCH /api/actions/{action_id}**

```
PATCH /api/actions/{action_id}  
Body: { status: 'accepted' | 'done' | 'ignored', ignoredReason?: string }  
  
Response: { success: boolean, action: ActionItem }
```

Backend logic

The Action Engine is currently implemented client-side in `src/lib/actionEngine.ts`. **This must be moved to the backend** so that:

1. Actions persist across sessions (currently they regenerate on every page load)
2. Actions can be generated by the nightly batch pipeline alongside forecasts
3. Action outcomes can be measured after the fact (did the promo actually generate +\$1,200?)

See Section 4 for the full engine spec.

Estimated complexity: L (new table, new API, new pipeline step)

4. ADDED: Action Engine (actionEngine.ts)

What it is

A rules-based recommendation engine that takes forecast data as input and generates a prioritized list of `ActionItem` objects. Currently runs entirely client-side as a prototype.

Action Types Generated

Type	Trigger	Example	Priority Logic
Labor (upstaff)	<code>laborPlan[day].recommendation === 'Upstaff'</code> AND <code>deltaHours > 2</code>	"Add staff for Saturday"	High if delta > 6h, Medium if > 3h
Labor (downstaff)	<code>laborPlan[day].recommendation === 'Downstaff'</code> AND <code>deltaHours < -3</code>	"Reduce hours on Tuesday"	Always Medium
Promo (event-driven)	Upcoming weather/seasonality event + top item without active promo	"Run promo for Heat Wave"	Always High
Promo (extend)	Item with active promo performing well	"Consider extending Coca-Cola promo"	Always Medium
Pricing (increase)	<code>todayVsTypical > 0</code> + high-volume item without promo	"Test price increase on Energy Drinks"	Always Low
Pricing (discount)	<code>todayVsTypical < -5</code> + item without promo	"Consider discount on Chips"	Always Medium
Event (weather)	Upcoming weather event in <code>insightEvents</code>	"Prepare for: Summer Heat Wave"	High if negative impact, Medium if positive
Event (local)	Upcoming seasonality event	"Back to School Rush"	Always High
Fuel	<code>fuelInsights[]</code> from convenience module	"Fuel: Regular tank below 20%"	Based on insight priority

Input Interface

```
interface ActionEngineInput {
  businessType: BusinessType;
  laborPlan: LaborPlanDay[];
  insightEvents: InsightEvent[];
  topItems: TopItem[];
  employees: Employee[];
  kpiData: KpiData;
  currentDate: string;
  fuelInsights?: FuelInsight[];
  fuelPrimaryDate?: string;
}
```

Value Estimation Logic (currently heuristic, needs ML refinement)

- **Labor upstaff:** $|\text{deltaHours}| \times \$18/\text{hour}$ (estimated revenue per labor hour)
- **Labor downstaff:** $|\text{deltaHours}| \times \$15/\text{hour}$ (hourly rate savings)
- **Promo event:** $\text{topItem.forecastRevenue} \times 0.12$ (12% lift estimate)
- **Promo extend:** $\text{item.forecastRevenue} \times 0.08$
- **Pricing increase:** $\text{forecastUnits} \times \text{priceIncrease} \times 0.7$ (assumes 30% volume loss)
- **Pricing discount:** $(\text{volumeLift} \times \text{price}) - (\text{forecastUnits} \times \text{discountAmount})$
- **Event:** $\text{revenueForecast} \times 0.03 \text{ to } 0.05$
- **Fuel:** Heuristic lookup by insight type (\$90–\$800)

What Nishant needs to build

1. **Move engine to backend.** Rewrite `actionEngine.ts` logic as a Python/Node service that runs:
 - **Nightly (after forecast pipeline):** Generate next-day actions for all stores
 - **Hourly (during open hours):** Refresh high-urgency actions (fuel tank levels, same-day labor)
2. **Replace heuristic value estimates with ML-calibrated ones:**
 - Use historical action outcomes (`ActionOutcome` table) to calibrate the expected value per action type

- Example: If "promo event" actions historically realize 9% lift (not 12%), adjust the multiplier
- This is a **Phase 2** improvement. For v1, the current heuristic multipliers are acceptable.

3. **Action deduplication:** Ensure the engine doesn't create duplicate actions for the same day/event if run multiple times.

4. **Action expiration:** Actions whose **dueDate** has passed without being acted on should auto-transition to **expired** status.

Estimated complexity: XL (core business logic, pipeline integration, eventual ML calibration)

5. ADDED: Action Next-Step Modal (Task Delegation)

What it is

When a user clicks "Take Action" on any action card, a **contextual modal** opens that adapts its UI to the action type:

Action Type	Modal Behavior
Labor (upstaff)	Shows suggested employee with name, phone, role. User can select a different employee from dropdowns. Pre-filled SMS message ("Can you cover Saturday 2-8pm?"). Click "Send" → simulates SMS dispatch. Shows "waiting for reply" state, then simulated confirmation.
Labor (downstaff)	Shows the overstaffed employee. Pre-filled message ("Shift on Tuesday has been reduced to 4 hours"). Option to offer the hours to another store.
Promo	Shows the related item, suggested discount %, and a date range. User can adjust discount and dates. Click "Launch Promo" → confirms.
Pricing	Shows current price, suggested new price, expected impact. User can adjust. Click "Apply Price" → confirms.
Event	Shows a preparation checklist (e.g., "Stock up beverages", "Schedule extra cashier"). Check off items and click "Ready".
Fuel	Shows tank levels, recommended order quantity, and tips. Click "Order Fuel" or "Adjust Price" depending on insight type.

Why it was built this way

- **One-click to action.** The user doesn't have to leave the dashboard to call an employee, create a promo, or change a price. Everything is handled inline.
- **SMS delegation** is huge for SMB. A store owner who sees "need 4 extra hours Saturday" can text their employee *directly from the dashboard*. This is the #1 "aha moment" in user testing.
- **Tailored per action type** so each modal shows exactly the right controls (no generic "confirm" dialog).

What Nishant needs to build

API: **POST /api/actions/{action_id}/execute**

```
POST /api/actions/{action_id}/execute
Body: {
  executionType: 'sms' | 'promo' | 'pricing' | 'checklist' | 'fuel_order',
  payload: {
    // For SMS:
    employeeId: string,
    message: string,
    // For Promo:
    skuId: string,
    discountPct: number,
    startDate: string,
    endDate: string,
    // For Pricing:
    skuId: string,
    newPrice: number,
    // For Fuel:
    grade: string,
    orderGallons: number
  }
}
```

Integrations required

Integration	Purpose	Provider	Priority
SMS Gateway	Send shift request / confirmation to employees	Twilio or Modisoft's existing SMS provider	P0 — core to labor actions
Promo Service	Create/update promo in Modisoft POS	Internal Modisoft API (already exists)	P0
Price Update Service	Push price changes to POS terminals	Internal Modisoft API (already exists)	P1
Fuel Ordering	Place fuel delivery orders	Depends on supplier integration	P2 — can be manual initially

Employee data source

The frontend currently uses a hardcoded **EMPLOYEES** map per business type. Backend needs:

```
GET /api/employees?store_id={store_id}
Response: Employee[] (id, name, phone, role, availability, maxHoursPerWeek, hourlyRa
```

This should pull from Modisoft's existing employee/scheduling module.

Estimated complexity: XL (multi-integration, SMS gateway, action execution pipeline)

6. ADDED: Quick Order Modal (Grocery/Retail)

What it is

When a user clicks "Order" on a category in the "Categories at Risk" widget, a modal opens showing:

- **Stockout alert context:** "Expected demand (450 units) exceeds current stock (180 units) for the next 48 hours."
- **Recommended order quantity:** Calculated as $(\text{expected} - \text{stock}) + (\text{expected} \times 0.20)$ (shortfall + 20% safety margin)
- **Supplier selector:** Primary Distributor (next day), Local Wholesaler (same day +15% cost), Central Warehouse (2-day)
- **Confirm Order button**

Why it was built this way

- **Pre-calculated quantity** removes guesswork. The store manager doesn't have to do math — the system tells them exactly how much to order based on the AI forecast.
- **Supplier choice** reflects real-world trade-offs: speed vs. cost. Same-day delivery costs more, but if you're running out in 8 hours, it's worth it.
- **Directly actionable from the alert.** No need to switch to a separate ordering system, open Excel, or call a distributor.

What Nishant needs to build

API: **POST /api/orders/quick**

```
POST /api/orders/quick
Body: {
  storeId: string,
  category: string,
  skuIds?: string[],
  quantity: number,
  supplierId: string,
  urgency: 'same_day' | 'next_day' | 'standard'
}
```

```
Response: {
  orderId: string,
  estimatedDelivery: string,
  totalCost: number,
  status: 'submitted' | 'pending_approval'
}
```

Data requirements

- **Supplier catalog:** Need a **suppliers** table mapping **store_id × category → supplier_id, name, lead_time_hours, cost_multiplier**.
- **Order quantity formula (backend-validated):**

```
shortfall = max(0, forecast_48h_units - current_stock)
safety_stock = forecast_48h_units × safety_margin_pct (default 20%, configurable)
recommended_qty = shortfall + safety_stock
```

The frontend sends the user's final quantity (they may adjust), but the backend should validate it's within reasonable bounds (e.g., not 10x the recommended).

Integration with existing PO system

The original PRD (Section 7.5) already defines `POST /api/po { lines:[{sku_id, qty}] }`. The Quick Order modal should call a **simplified wrapper** around this existing endpoint that:

1. Automatically expands a category-level order into individual SKU lines based on the category's demand distribution
2. Selects the supplier based on the user's choice
3. Returns a confirmation with estimated delivery

Estimated complexity: M (wraps existing PO endpoint + supplier logic)

7. ADDED: Substitute Modal (Grocery/Retail)

What it is

When a user clicks "Substitute" on a category in the "Categories at Risk" widget, a modal opens showing:

- **Context:** "Select an in-stock alternative to promote while waiting for the next Dairy delivery."
- **2–3 AI-suggested alternatives** with:
 - Product name
 - Current stock level
 - Match score (0–100%) indicating how similar the substitute is to the out-of-stock products
- **Radio selection** to pick the preferred substitute
- **"Promote Substitute" button** to confirm

Why it was built this way

- **Captures revenue that would otherwise walk out the door.** When a customer comes for organic milk and it's out, they might buy standard milk if it's prominently displayed. This modal helps the manager make that call quickly.

- **Match score** builds trust. The manager can see that "Standard Whole Milk" is a 92% match, vs. "Oat Milk" at 75%, and decide accordingly.
- **Complements the Order action.** Order fixes the problem in 24–48 hours. Substitute fixes it *right now* by capturing today's demand with what's currently in stock.

What Nishant needs to build

API: **GET /api/inventory/substitutes**

```
GET /api/inventory/substitutes?store_id={store_id}&category={category}
```

```
Response: {
  substitutes: [
    {
      skuId: string,
      name: string,
      currentStock: number,
      matchScore: number,
      matchReason: string,
      estimatedDemandCapture: number
    }
  ]
}
```

AI / ML for match scoring

The match score is the most important part of this feature. It should be calculated based on:

1. **Category proximity:** Same subcategory = high base score
2. **Price similarity:** Within $\pm 20\%$ of the out-of-stock item's price = bonus
3. **Historical substitution data:** If customers who bought Product A also bought Product B (market basket analysis), that's the strongest signal
4. **Current stock levels:** Only suggest items with stock > minimum threshold

Phase 1 (v1) — Rules-based scoring:

```
base_score = 50 (same department) or 80 (same subcategory) or 95 (same brand family)
price_adjustment = -20 if price_diff > 30% else 0
stock_adjustment = -10 if stock < 50 units else 0
match_score = clamp(base_score + price_adjustment + stock_adjustment, 0, 100)
```

Phase 2 (v2) — ML-powered: Use collaborative filtering on POS transaction data to find actual purchase substitution patterns. This would use historical co-purchase data to generate a **substitution_affinity** matrix.

API: **POST /api/inventory/promote-substitute**

```
POST /api/inventory/promote-substitute
Body: {
  storeId: string,
  originalCategory: string,
  substituteSkuId: string,
  action: 'shelf_placement' | 'price_tag' | 'digital_signage'
}
```

This logs the substitution decision for tracking. In the future, it could trigger digital signage updates, POS upsell prompts for cashiers, or shelf label printing.

Estimated complexity: M (Phase 1 rules-based), L (Phase 2 with ML affinity)

8. ADDED: New Item Simulator

What it is

A full-screen modal (triggered by the "Test New Item" button in top nav) that lets an owner simulate introducing a new product before committing to ordering it. Includes:

Input form:

- Item name, price, category
- Store selection
- Promo toggle (with discount %)

AI-generated projections (displayed instantly):

- Daily / weekly / monthly units and revenue
- Cannibalization % (how much it steals from existing items)
- Net new revenue (after cannibalization)
- Break-even days
- Confidence score (0–100)
- 30-day demand curve chart (area chart with projected ramp-up and stabilization)
- 3–4 AI-generated insights (e.g., "Similar items in this category sell 45 units/day")

Category benchmarks: Pre-loaded averages for each category (avg price, avg daily units, price elasticity)

Why it was built this way

- **Reduces risk for new products.** SMB owners often add new items based on gut feeling or distributor sales pitches. This lets them see a data-driven projection *before* spending money on inventory.
- **Cannibalization is the key insight.** A new energy drink might look great in isolation, but if it just steals sales from your existing energy drinks, the net gain is much smaller. The simulator makes this visible.
- **Category benchmarks** provide context even when the owner hasn't decided on exact parameters yet.

What Nishant needs to build

API: **POST** `/api/simulator/new-item`

```
POST /api/simulator/new-item
```

```
Body: {
```

```
  storeId: string,  
  name: string,  
  price: number,  
  category: string,  
  isPromo: boolean,  
  promoDiscountPct: number
```

```
}
```

```
Response: {
```

```
  projection: {  
    dailyUnits: number,  
    dailyRevenue: number,  
    weeklyUnits: number,  
    weeklyRevenue: number,  
    monthlyUnits: number,  
    monthlyRevenue: number,  
    cannibalizationPct: number,  
    netNewRevenue: number,  
    breakEvenDays: number,  
    confidenceScore: number,  
    demandCurve: [{ day: number, units: number, revenue: number }],  
    insights: string[]
```

```
  },
```

```
  benchmarks: {  
    avgPrice: number,  
    avgDailyUnits: number,  
    elasticity: number
```

```
  }
```

```
}
```

ML / AI Requirements

Component	Approach	Model	Priority
Demand projection	Find similar items in same store+category, apply adoption curve	LightGBM (existing) + analogy-based method	P1
Cannibalization estimate	Cross-elasticity from historical data when new items were added	Regression on historical new-item launches	P2
Break-even calculation	$(\text{initial_order_cost}) / (\text{daily_net_revenue})$	Deterministic (no ML)	P0
Demand curve shape	30-day ramp: novelty peak (days 1–5), dip (days 6–14), stabilization (days 15–30)	Parameterized curve fitted to category history	P1
AI Insights	Category benchmarks + comparison to similar items	GPT-4 mini (batch) or rules-based templates	P1

Phase 1 approach (no new ML):

1. Compute avg_daily_units and avg_price for same category in same store
2. Apply price elasticity: $\text{projected_units} = \text{benchmark_units} \times (\text{benchmark_price} / \text{input_price})^{\text{elasticity}}$
3. Apply promo boost if applicable: $\text{projected_units} \times (1 + \text{promoLift})$
4. Cannibalization: Use a flat rate per category (e.g., 15% for Beverages, 25% for Snacks)
5. Demand curve: Use a parameterized S-curve with category-specific coefficients

Phase 2 approach (ML-powered):

- Train a "new item forecaster" model on historical new-item introduction data across all Modissoft tenants
- Input features: category, price_vs_category_avg, store_size, store_location_type, day_of_week_launched, is_promo
- Output: 30-day unit forecast + confidence interval

Estimated complexity: L (Phase 1 rules-based), XL (Phase 2 cross-tenant ML)

9. ADDED: Fuel Forecasting Module (Convenience)

What it is

A comprehensive fuel/gas station overlay for the Convenience business type. Includes:

- **Fuel KPI cards:** Gallons today, fuel revenue, fuel margin, avg price/gallon

- **Tank level visualization:** Per-grade (Regular, Plus, Premium, Diesel) with fill-level bars, capacity, and reorder threshold alerts
- **Fuel demand forecast chart:** Hourly demand curve showing peak hours and rush periods
- **Fuel-specific AI insights** (tank low, rush hour staffing, cross-sell opportunities, weather-driven demand, price alerts)
- **Fuel actions** integrated into the main Action Center (type: 'fuel')

Fuel Insight Types

Type	Description	Example
tank_low	Tank below reorder threshold	"Regular Unleaded 1 below 20% — schedule delivery"
rush_hour	High-demand period approaching	"Peak demand 7–9am — ensure pumps staffed"
cross_sell	Fuel-to-store conversion opportunity	"32% of fuel customers buy inside — promote coffee"
price_alert	Competitor price change or margin opportunity	"Competitor dropped Regular by \$0.05"
weather	Weather impact on fuel demand	"Rain forecast — expect 15% fewer gallons"
event	Local event affecting traffic	"High school football tonight — expect +20% evening traffic"

Why it was built this way

- **Fuel is 60–70% of revenue for convenience stores** but the original PRD focused primarily on in-store merchandise. This module fills a massive gap.
- **Cross-sell insights** bridge fuel and in-store. The insight "32% of fuel customers buy inside" directly ties to the core Modisoft value prop of growing basket size.
- **Tank level monitoring** prevents the most expensive problem: running out of fuel during peak hours.

What Nishant needs to build

Data Sources

Source	Data	Frequency
Tank monitoring system (ATG)	Current level per tank, capacity, grade	Real-time (via Modisoft POS or Veeder-Root integration)
POS fuel transactions	Gallons, grade, price, timestamp, transaction_id	Real-time
Fuel supplier/delivery	Delivery schedules, order history	Daily
Competitor pricing	Nearby station prices (if available)	Daily (GasBuddy API or manual entry)

API: **GET /api/fuel/status**

```
GET /api/fuel/status?store_id={store_id}
```

```
Response: {
  tanks: FuelTank[],
  todayForecast: FuelDayForecast,
  insights: FuelInsight[],
  sevenDayForecast: FuelDayForecast[]
}
```

API: **GET /api/fuel/forecast**

```
GET /api/fuel/forecast?store_id={store_id}&horizon={7|14|28}
```

```
Response: {
  daily: FuelDayForecast[],
  totalGallons: number,
  totalRevenue: number,
  peakDay: string,
  avgConversionRate: number
}
```

ML Model for Fuel Demand

- **Model:** LightGBM (can share infrastructure with the existing demand model)
- **Granularity:** per store × grade × hour
- **Features:** Day of week, hour of day, temperature, precipitation, gas price (own + competitor delta), holiday/event flags, historical gallons sold (lag features)
- **Training:** Nightly, alongside the main demand model
- **Output:** Hourly gallon forecast per grade for next 7–28 days

Estimated complexity: XL (new data integration, new ML model, tank monitoring)

10. EXISTING (Updated): Categories at Risk Widget

What changed

The "Categories at Risk (next 48h)" widget from the original PRD (Section 6.3 — Grocery/Retail) now has **fully functional action buttons**:

- **"Order" button** → Opens the Quick Order Modal (Section 6)
- **"Substitute" button** → Opens the Substitute Modal (Section 7)

Previously these were static/non-functional buttons in the PRD wireframe.

Additional context

The widget data is currently hardcoded in the frontend:

```
Dairy:    expected=450, stock=180, hoursLeft=12, badge='Velocity-based'  
Produce: expected=380, stock=95,  hoursLeft=8,  badge='Inventory-based'  
Bakery:   expected=220, stock=45,  hoursLeft=6,  badge='Inventory-based'
```

This must be replaced with live data from the backend.

API (already defined in original PRD, needs implementation)

```
GET /api/widgets?type=risk&store_id={store_id}&dept=grocery
```

```
Response: {
  categories: [
    {
      category: string,
      expected48h: number,
      stockCanSell: number,
      hoursLeft: number,
      badge: 'Inventory-based' | 'Velocity-based',
      skuBreakdown: [
        { skuId: string, name: string, expected: number, stock: number }
      ]
    }
  ]
}
```

Badge logic (per original PRD Section 9.2):

- **Inventory-based** = inventory snapshot is fresh ($\text{last_count_dt} \leq 24\text{h}$) and sane
- **Velocity-based** = inventory is stale or unreliable; `hours_left` computed from recent sell rate only

Estimated complexity: M (API exists in spec, needs implementation + inventory integration)

11. New API Contracts (Summary)

Endpoint	Method	Section	Priority	Notes
/api/actions	GET	3	P0	Core feature
/api/actions/{id}	PATCH	3	P0	Status updates
/api/actions/{id}/execute	POST	5	P0	Action delegation
/api/employees	GET	5	P0	Pull from existing HR module
/api/orders/quick	POST	6	P1	Wraps existing PO endpoint
/api/inventory/substitutes	GET	7	P1	Match scoring
/api/inventory/promote-substitute	POST	7	P2	Logging + future automation
/api/simulator/new-item	POST	8	P1	Projection engine
/api/fuel/status	GET	9	P1	Convenience only
/api/fuel/forecast	GET	9	P1	Convenience only
/api/widgets?type=risk	GET	10	P0	Already in original PRD

All endpoints must follow the existing RLS/tenant-safety pattern from PRD Section 7.5.

12. New Data Models / Tables

actions table

```
CREATE TABLE actions (  
  id                UUID PRIMARY KEY,  
  tenant_id         UUID NOT NULL,  
  store_id          VARCHAR(50) NOT NULL,  
  type              VARCHAR(20) NOT NULL,  
  priority          VARCHAR(10) NOT NULL,  
  title             TEXT NOT NULL,  
  description        TEXT NOT NULL,  
  expected_value     DECIMAL(10,2),  
  expected_value_label TEXT,  
  due_date          DATE NOT NULL,  
  status            VARCHAR(20) DEFAULT 'open',  
  owner_role        VARCHAR(20),  
  related_item       VARCHAR(255),  
  related_category   VARCHAR(100),  
  suggested_employee_id UUID,  
  created_at         TIMESTAMP DEFAULT NOW(),  
  accepted_at        TIMESTAMP,  
  completed_at       TIMESTAMP,  
  ignored_reason     TEXT,  
  execution_payload  JSONB,  
  FOREIGN KEY (tenant_id) REFERENCES tenants(id)  
);  
  
CREATE INDEX idx_actions_store_date ON actions(store_id, due_date);  
CREATE INDEX idx_actions_status ON actions(status);
```

action_outcomes table

```
CREATE TABLE action_outcomes (  
  id                UUID PRIMARY KEY,  
  action_id         UUID NOT NULL REFERENCES actions(id),  
  expected_value    DECIMAL(10,2),  
  realized_value    DECIMAL(10,2),  
  notes            TEXT,  
  measured_at       TIMESTAMP,  
  measurement_method VARCHAR(50)  
);
```

substitution_log table

```
CREATE TABLE substitution_log (  
  id                UUID PRIMARY KEY,  
  tenant_id         UUID NOT NULL,  
  store_id          VARCHAR(50) NOT NULL,  
  original_category VARCHAR(100),  
  original_sku_ids  TEXT[],  
  substitute_sku_id VARCHAR(50),  
  action_taken       VARCHAR(50),  
  created_at        TIMESTAMP DEFAULT NOW(),  
  demand_captured_pct DECIMAL(5,2)  
);
```


fuel_tanks table

```
CREATE TABLE fuel_tanks (  
  id                UUID PRIMARY KEY,  
  store_id          VARCHAR(50) NOT NULL,  
  grade             VARCHAR(20) NOT NULL,  
  label             VARCHAR(100),  
  capacity_gallons  INTEGER,  
  current_level     INTEGER,  
  price_per_gallon  DECIMAL(5,3),  
  profit_per_gallon DECIMAL(5,3),  
  reorder_threshold INTEGER,  
  last_delivery     TIMESTAMP,  
  next_delivery     TIMESTAMP,  
  updated_at        TIMESTAMP DEFAULT NOW()  
);
```

quick_orders table

```
CREATE TABLE quick_orders (  
  id                UUID PRIMARY KEY,  
  tenant_id         UUID NOT NULL,  
  store_id          VARCHAR(50) NOT NULL,  
  category          VARCHAR(100),  
  quantity          INTEGER NOT NULL,  
  supplier_id       VARCHAR(50),  
  urgency           VARCHAR(20),  
  status            VARCHAR(20) DEFAULT 'submitted',  
  po_id            UUID,  
  estimated_delivery TIMESTAMP,  
  total_cost        DECIMAL(10,2),  
  created_at        TIMESTAMP DEFAULT NOW()  
);
```

13. AI / ML Requirements

Models

Model	Purpose	Input	Output	Training	Infra
LightGBM (existing)	Revenue & unit forecast per store×SKU×day	POS, weather, promos, calendar	<code>forecast_units</code> , <code>forecast_revenue</code>	Nightly	Existing
LightGBM (fuel)	Gallon forecast per store×grade×hour	Fuel POS, weather, price, traffic	<code>forecast_gallons</code> per hour	Nightly	New pipeline same
GPT-4 mini (existing)	Parse promos, generate insights	Promo text, forecast context	Structured fields, insight strings	Batch (nightly)	Existing
GPT-4 mini (new item)	Generate insights for new item simulator	Category benchmarks, item params	3–4 insight strings	On-demand	Existing minor
Rules engine (actions)	Generate action recommendations	Forecast, labor plan, events, KPIs	<code>ActionItem[]</code>	N/A (deterministic)	New service
Rules engine (substitutes)	Score substitute products	Category tree, price, stock	<code>match_score</code> per candidate	N/A (deterministic)	New service
Collaborative filtering (Phase 2)	Improve substitute scoring	POS co-purchase data	<code>substitution_affinity</code> matrix	Weekly	New n
Action value calibration (Phase 2)	Improve expected value estimates	Historical <code>action_outcomes</code>	Calibrated multipliers	Monthly	Lightw

LLM Usage & Cost

Use Case	Model	Trigger	Est. Tokens/Call	Est. Cost/Store/Day
Promo parsing	GPT-4 mini	Nightly batch	~500	\$0.001
Insight generation	GPT-4 mini	Nightly batch	~800	\$0.002
New item insights	GPT-4 mini	On-demand	~600	\$0.001 (amortized)
Ask Sunny chat	GPT-4 (full)	User-initiated	~2,000	Gated behind click
Total estimated				< \$0.005/store/day

14. Third-Party Services & Tools

Service	Purpose	Current Status
Twilio (or Modisoft SMS)	Send SMS for labor action delegation	Required for v1
OpenAI API (GPT-4 mini)	Insight generation, promo parsing, new item insights	Already integrated
Weather API	Temperature, precipitation for demand adjustments	Already integrated
GasBuddy / OPIS API	Competitor fuel pricing	Nice-to-have
Veeder-Root / ATG	Real-time tank level monitoring	Required for fuel module
Next.js 16	Frontend framework	In use
LightGBM	Forecast model	Already in use
Prefect / Azure Functions	Pipeline orchestration	Already planned

15. Updated Acceptance Criteria

#	Criterion	Test
AC-1	Action Center renders inline on dashboard with correct action count badge	Load dashboard → see "Suggested Actions Based on Forecast" with badge
AC-2	Clicking "Take Action" on a labor action opens the correct modal with employee data	Click → verify SMS form with suggested employee pre-filled
AC-3	Clicking "Take Action" on a promo action opens promo configuration modal	Click → verify item name, discount %, date range
AC-4	Clicking "Order" on a Categories at Risk row opens Quick Order Modal	Click → verify shortfall math is correct
AC-5	Quick Order recommended quantity = (expected – stock) + 20% safety	Dairy: $(450 - 180) + (450 \times 0.2) = 270 + 90 = 360$
AC-6	Clicking "Substitute" opens Substitute Modal with 2+ alternatives	Click Dairy → see Standard Whole Milk and Oat Milk
AC-7	Substitute match scores are reasonable (same subcategory > cross-category)	Standard Whole Milk > 85%, Oat Milk < 80%
AC-8	New Item Simulator opens from "Test New Item" button	Click → full modal with input form and projections
AC-9	Simulator projections update instantly when inputs change	Change price → see revenue/units recalculate
AC-10	Fuel module shows tank levels for Convenience business type only	Switch to Convenience → see fuel; Switch to Grocery → no fuel
AC-11	Action status persists across page loads (when backend is connected)	Accept an action → refresh → action still shows as "accepted"
AC-12	Actions auto-sort by priority (High → Medium → Low), then by expected value	Verify High-priority actions always appear first

16. Migration Notes

For Nishant — Getting Started

1. Start with the **actions** table and **GET /api/actions** endpoint. This unblocks the entire Action Center. The frontend is already rendering mock actions from the client-side engine; swapping to a real API is the highest-leverage change.

2. **Port `actionEngine.ts` to the backend.** The logic is clean and well-documented in the source file. It's ~500 lines of TypeScript that can be rewritten in Python or kept in Node. The key functions are:
 - `generateLaborActions()` — needs labor plan data
 - `generatePromoActions()` — needs top items + insight events
 - `generatePricingActions()` — needs top items + KPI data
 - `generateEventActions()` — needs insight events + KPI data
 - `generateFuelActions()` — needs fuel insights (convenience only)
 - `buildActions()` — orchestrator that combines and sorts all actions
3. **SMS integration is the highest-impact demo feature.** If we can show an owner texting an employee directly from the dashboard to cover a shift, that's the "wow" moment.
4. **Fuel module can be done in parallel** since it's isolated to convenience stores. If Veeder-Root/ATG integration is too complex for v1, start with manual tank level entry.
5. **New Item Simulator can use rules-based projections for v1.** The category benchmarks are already hardcoded in the frontend. Backend just needs to pull real benchmarks from POS history.

Recommended Sprint Sequence

Sprint	Focus	APIs	Est. Days
1	Actions table + GET/PATCH actions API + Backend action engine	Actions CRUD	5–7
2	Action execution + SMS integration (Twilio)	Execute endpoint + Twilio	5–7
3	Categories at Risk API (live data) + Quick Order (wraps PO)	Risk widget + Quick Order	3–5
4	Substitute API (rules-based scoring)	Substitute GET + POST	3–4
5	New Item Simulator API (rules-based projections)	Simulator POST	3–5
6	Fuel module (tank data + LightGBM fuel model)	Fuel status + forecast	7–10
7	Action outcome measurement + value calibration	Outcomes table + auto-measurement	5–7

Total estimated: 31–45 dev days

17. Change Log Entry

Add to original PRD Section 16:

v1.3 — February 27, 2026 — Major UX overhaul and feature additions based on SMB user testing.

Removed:

- Lock Plan / Execution Board (too complex for SMB persona)
- Scenario Inputs & Multiplier Engine (replaced by direct actions)
- Redundant Action Center drawer (merged into inline dashboard widget)

Added:

- Inline Action Center with AI-generated task recommendations (labor, promo, pricing, event, fuel)
- Action Engine (actionEngine.ts) — rules-based recommendation system
- Action Next-Step Modal — contextual task execution with SMS delegation for labor actions
- Quick Order Modal (Grocery/Retail) — one-click ordering from Category Risk alerts
- Substitute Modal (Grocery/Retail) — AI-scored alternative product suggestions
- New Item Simulator — data-driven new product introduction projections
- Fuel Forecasting Module (Convenience) — tank monitoring, hourly demand, fuel-specific insights

Updated:

- Categories at Risk widget now has fully functional Order and Substitute buttons
- Action types expanded: labor | promo | pricing | event | fuel
- 10 new API endpoints defined (see Section 11)
- 5 new database tables defined (see Section 12)
- Sprint plan for backend implementation (31–45 dev days)

End of PRD Addendum v1.3